

Ad: Tuğba Nur

Soyad: Gültekin

Numara: 25360859065

Bölüm: Bilgisayar Mühendisliği

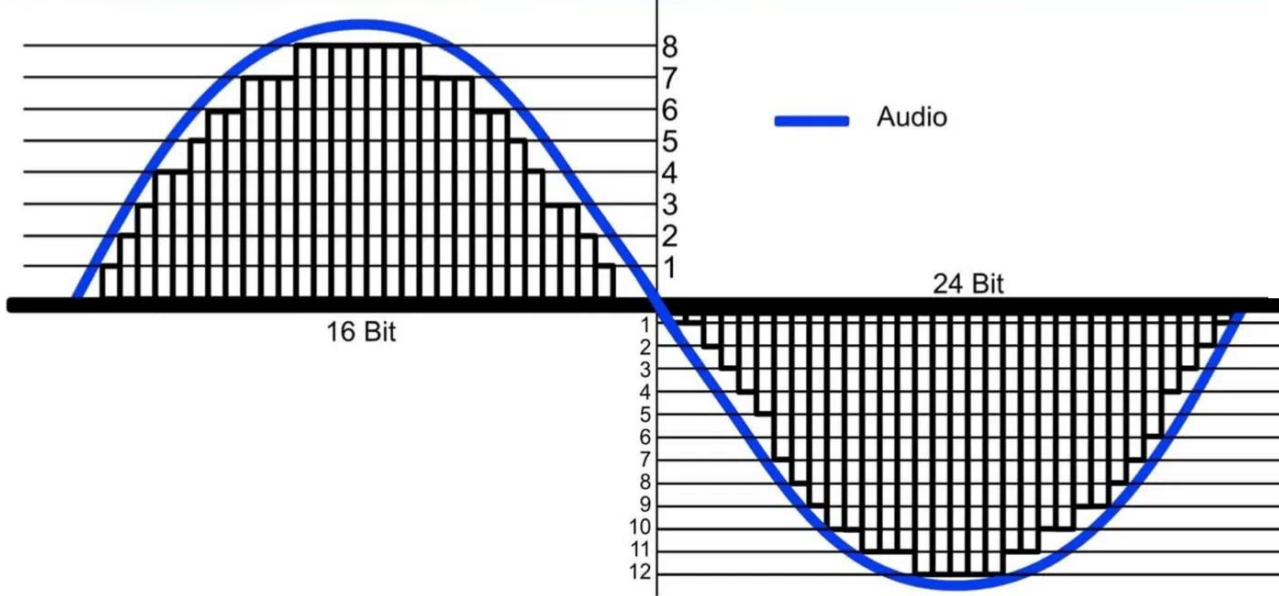
Konu: Veri Depolama ve Sayısal Sistemler / 1. Proje

Tarih: 20/12/2025

Veri Depolama ve Sayısal Sistemler

Bilgisayarın içinde bilgiler nasıl kodlanır ve depolanır?

- Bugünün bilgisayarları içinde bilgi 0'lar ve 1'ler şeklinde kodlanmaktadır. Bu rakamlar **bitler** (*ikilik rakamlar için kısaltma*) olarak adlandırılmaktadırlar. Sayısal değerler ile bit'leri ilişkilendirme eğiliminde olmanıza rağmen onlar, gerçekten anlamları eldeki uygulamaya bağlı olan sembollerdir sadece. Bilgisayarlar için her şey (sayı, harf, resim, video, ses) en sonunda bitlere indirgenir.



Decimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Bit nedir?

- Bit (Binary Digit) bilgisayardaki **en küçük veri birimidir**. Sadece iki değer alabilir:
 - 0 → Kapalı / Yanlış / Akım yok
 - 1 → Açık / Doğru / Akım var

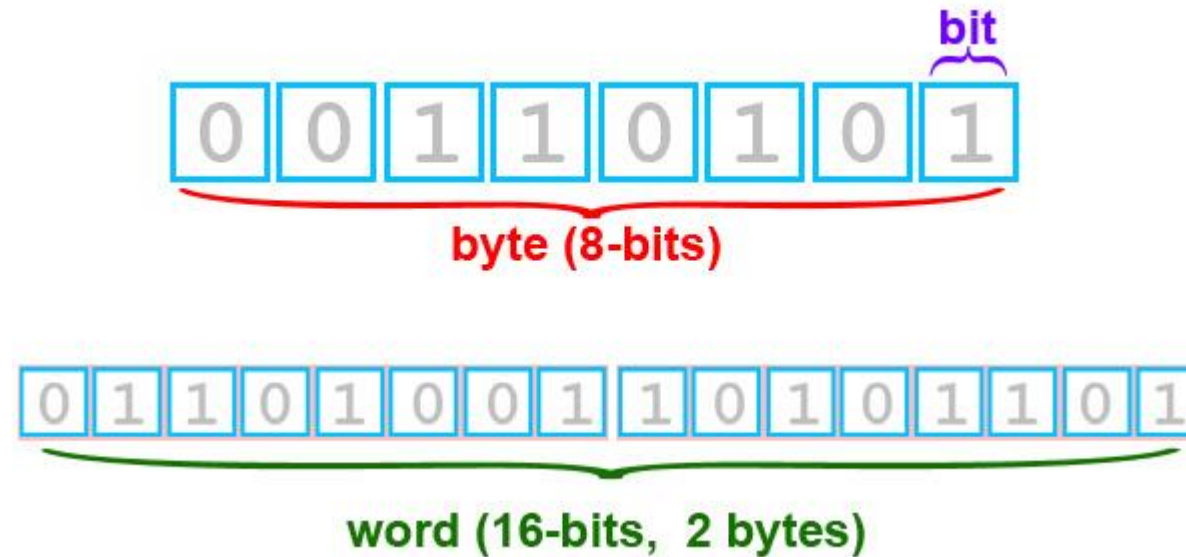
Bilgisayarın ikilik sistemle çalışmasının sebebi:

- Elektronik devrelerde iki kararlı durum vardır (açık-kapalı).
- Bu durumlar gürültüye dayanıklıdır ve güvenilirdir.



Byte nedir?

- Elektronik ve bilgisayar bilimlerinde genellikle 8 bitlik dizilim boyunca 1 veya 0 değerlerini bünyesine alan ve kaydedebilen bilgilerin türünden bağımsız bir bellek ölçüm birimidir. Bir byte, latin alfabesini baz alan 8-bitlik bir kodlamada herhangi bir harfi temsil eder.



- ‘Çoğu platformda’ 8-bit=1 byte
- Bitten sonraki ikinci en küçük sayısal bilgisayar bilimidir. Bir byte, 0 ile 255 arasındaki değeri veya diğer anlamda 256 şalter durumunu temsil etmektedir.
- 2^8 bitin onluk sayı değeri 255 olup, 0 ile birlikte, 256 şalter durumunu gösterir. Eğer somut sonuç 2 üzeri 10’u geçiyorsa o zaman sayının sonundaki rakamlar silinip onun yerine kısaltmalar eklenir.

Multiples of bytes v · d · e				
SI decimal prefixes		Binary usage	IEC binary prefixes	
Name (Symbol)	Value		Name (Symbol)	Value
kilobyte (kB)	10^3	2^{10}	kibibyte (KiB)	2^{10}
megabyte (MB)	10^6	2^{20}	mebibyte (MiB)	2^{20}
gigabyte (GB)	10^9	2^{30}	gibibyte (GiB)	2^{30}
terabyte (TB)	10^{12}	2^{40}	tebibyte (TiB)	2^{40}
petabyte (PB)	10^{15}	2^{50}	pebibyte (PiB)	2^{50}
exabyte (EB)	10^{18}	2^{60}	exbibyte (EiB)	2^{60}
zettabyte (ZB)	10^{21}	2^{70}	zebibyte (ZiB)	2^{70}
yottabyte (YB)	10^{24}	2^{80}	yobibyte (YiB)	2^{80}

Boolean İşlemleri

- 1 / 0 olarak atadığımız doğru / yanlış değerlerini değiştiren işlemlere, mantık adı verilen matematik alanının öncüsü olan matematikçi George Boole'nin (1815-1864) anısına, Boolean işlemleri denilmektedir.
- Boolean İşlemleri, dijital sistemlerin ve bilgisayarın **temel mantık yapısıdır.**



Temel Boolean İşlemleri

- ✓ AND (VE), OR (VEYA), XOR (ÖZEL VEYA). Bu işlemler TIME (KERE) ve PLUS (TOPLAM) aritmetik işlemlerine benzemektedirler, çünkü üçüncü bir değer (çıktı) üretmek için üretmek için bir çift değeri (işlemin girdileri) birleştirmektedirler. Ancak, aritmetik değerlerin aksine Boolean işlemleri sayısal değerler yerine doğru/yanlış değerlerini birleştirmektedir.
- ✓ NOT (DEĞİL) işlemi bir diğer boolean işlemidir. AND, OR, XOR'dan farklıdır, çünkü sadece bir girişe sahiptir. Çıkışı girişin tersidir; eğer NOT işleminin girişi doğru ise, çıkışı yanlıştır ve tam tersi de gerçekleşmektedir.

The AND operation

$$\begin{array}{r} 0 \\ \text{AND } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ \text{AND } 1 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1 \\ \text{AND } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1 \\ \text{AND } 1 \\ \hline 1 \end{array}$$

The OR operation

$$\begin{array}{r} 0 \\ \text{OR } 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ \text{OR } 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{OR } 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{OR } 1 \\ \hline 1 \end{array}$$

The XOR operation

$$\begin{array}{r} 0 \\ \text{XOR } 0 \\ \hline 0 \end{array}$$

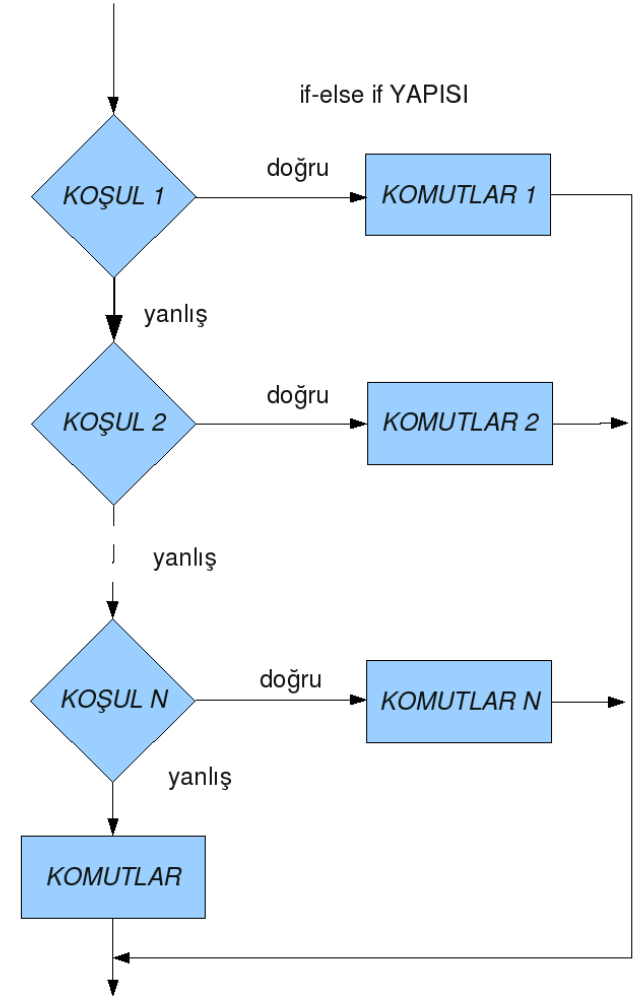
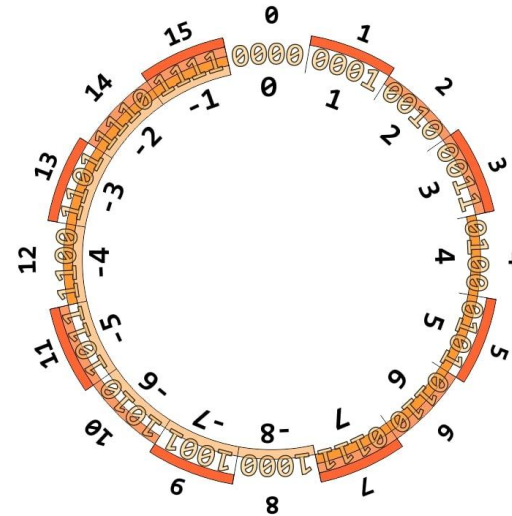
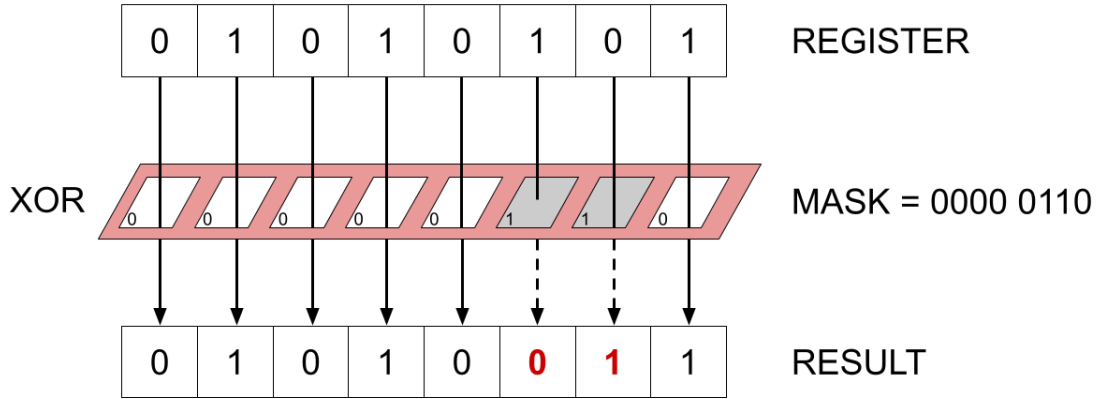
$$\begin{array}{r} 0 \\ \text{XOR } 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{XOR } 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ \text{XOR } 1 \\ \hline 0 \end{array}$$

Boolean İşlemlerinin Kullanım Alanlar

- ✓ CPU aritmetik işlemleri
- ✓ Karar mekanizmaları (if-else)
- ✓ Bit maskeleye (bit masking)
- ✓ Two's Complement
- ✓ Dijital devre tasarımı



Mantık Kapısı

AND



Inputs	Output
0 0	0
0 1	0
1 0	0
1 1	1

OR



Inputs	Output
0 0	0
0 1	1
1 0	1
1 1	1

XOR



Inputs	Output
0 0	0
0 1	1
1 0	1
1 1	0

NOT

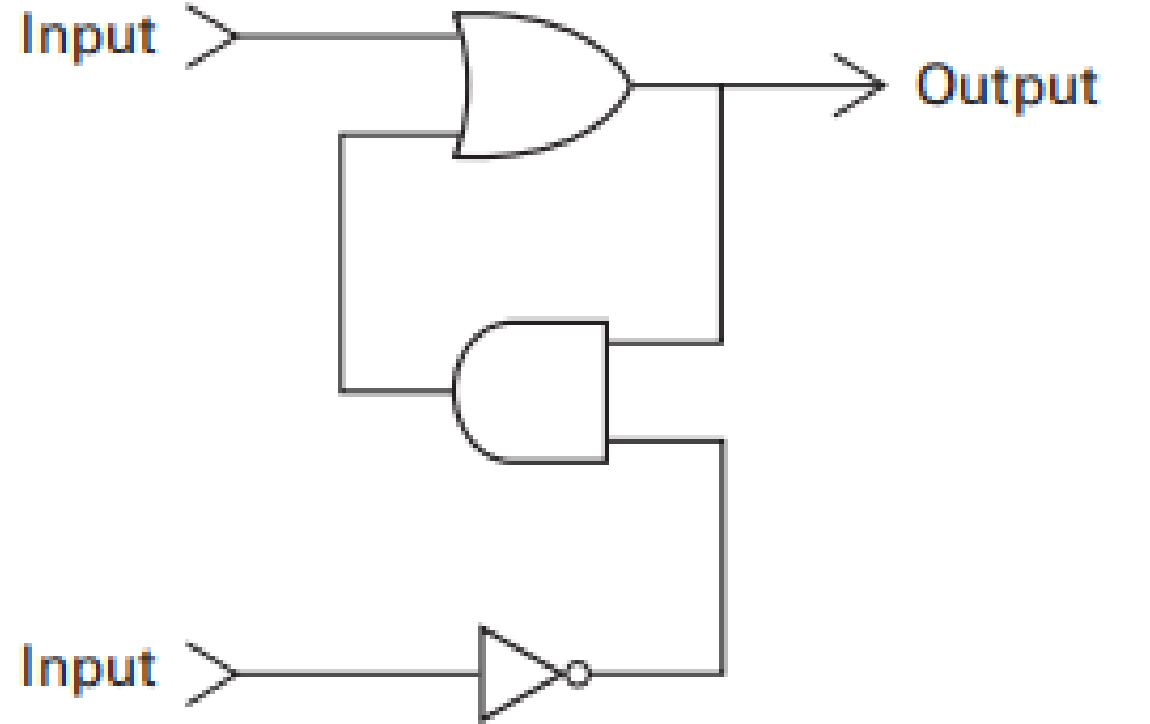


Inputs	Output
0	1
1	0

- Boolean işlemini donanımda yapan devredir. Verilen giriş değerlerine göre çıkışını üreten ayardır. Kapılar dişliler, röleler, ve optik aygıtlar gibi çeşitli teknolojilerden imal edilebilmektedir. Bugünkü bilgisayarların içinde, kapılar genellikle voltaj d-seviyeleri olarak gösterilen 0 ve 1 basamaklarının küçük elektronik devreler olarak uygulanmaktadır. AND, OR, XOR ve NOT kapılarının, bir taraftan giriş değerleri ve diğer taraftan çıkış değeri ile belirgin olarak şekillendirilmiş semboller tarafından gösterildiğine dikkat edilmelidir.

Flip-Flop (Yaz-Boz)

- Dijital elektronik devrelerde 1 bitlik bilgiyi saklayabilen **temel bellek elemanıdır**. Yani yalnızca 0 veya 1 değerini tutar ve bu değeri kontrol sinyallerine göre değiştirir.
- Belleklidir (**hafızası vardır**).
- Çıkışı, yalnızca girişe değil önceki durumuna da bağlıdır. Bu yüzden flip-floplar **ardışıl (sequential)** devrelerin temel yapı taşıdır.

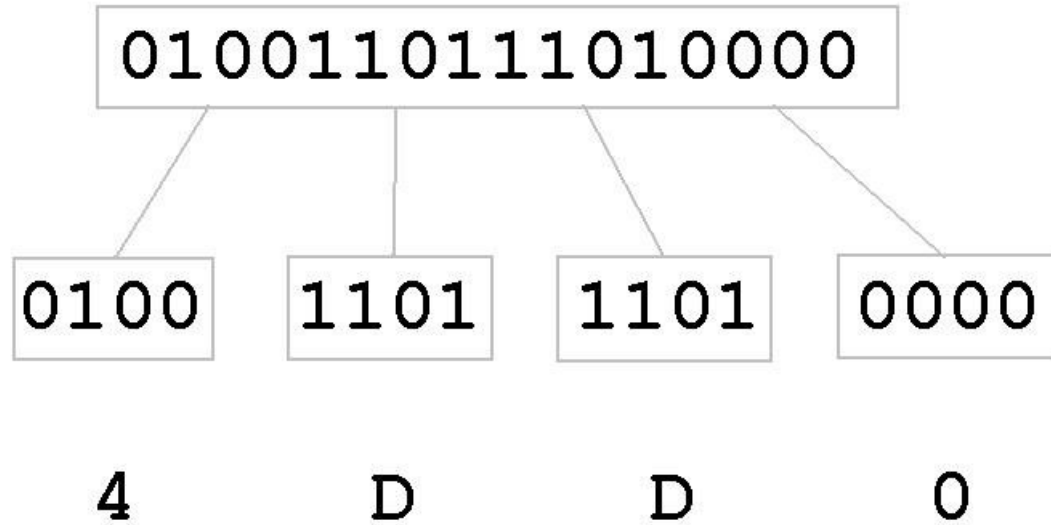


Onaltılık Gösterim

- Bilgisayarlar ikili (binary) sistemle çalışır. Ancak ikili sayılar çok uzun dizgelidirler. Bitlerin uzun dizgeleri genellikle **akım (stream)** olarak adlandırılmaktadır. Ne yazık ki, insanların akımları okuyup anlayabilmesi güçtür. Bu uzun ikilik gösterimleri basitleştirmek için, biz genellikle makinedeki bitlerin dördün katları uzunluklarına sahip olma avantajlarından yararlanan, ikili sayıların kısa ve okunabilir hâli olan **onaltılık gösterim (hexadecimal notation)** denir.

Denary	Binary	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Hexadecimal Sistemi Binary Sisteme Çevirme

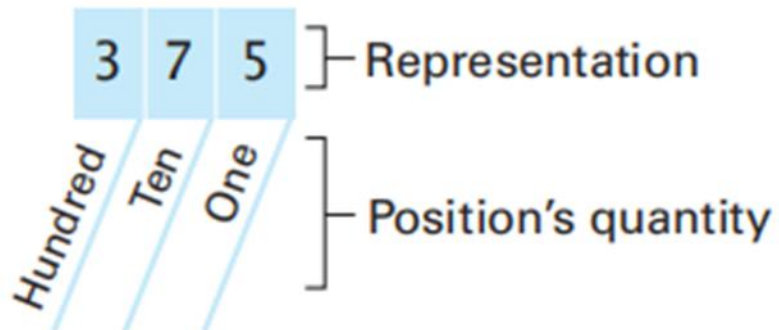


- Bu bit desenini dört bit uzunlukta alt dizgelere bölerek ve her dizgeyi onaltılık eşitliği ile göstererek elde edilmektedir.
 - ✓ 0000 → 0
 - ✓ 1101 → D
 - ✓ 1101 → D
 - ✓ 0100 → 4
- Bu şekilde, 16 bit desen 0100110111010000, 4DD0 olarak daha kabul edilebilir şekilde indirgenebilmektedir.

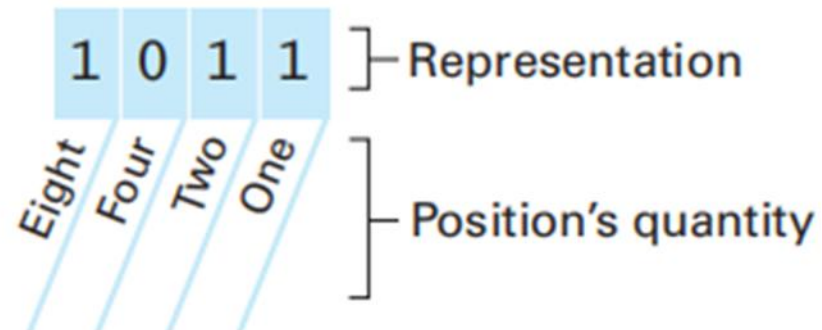
İkilik Sistem

- 10 tabanlı sistemini hatırlayarak, bir gösterimdeki her pozisyon bir büyüklük ile ilişkilidir. 375'in gösteriminde; 5 bir büyüklüğü ile ilgili bir pozisyondadır ve 3 yüz büyüklüğü ile ilgili bir pozisyondadır. Her büyüklük sağındaki büyüklüğün 10 katıdır. İlgili ifadenin gösterdiği değer, rakamın pozisyonu ile ilgili büyüklüğün basamak değeri ile çarpılması ve bu çarpımların toplanması ile elde edilmektedir. Örnekle açıklamak için 375 deseni $(3 \times \text{yüz}) + (7 \times \text{on}) + (5 \times \text{bir})$ ile daha da teknik olarak ifade etmek gerekirse $(3 \times 10^2) + (7 \times 10^1) + (5 \times 10^0)$ ile gösterilmektedir.

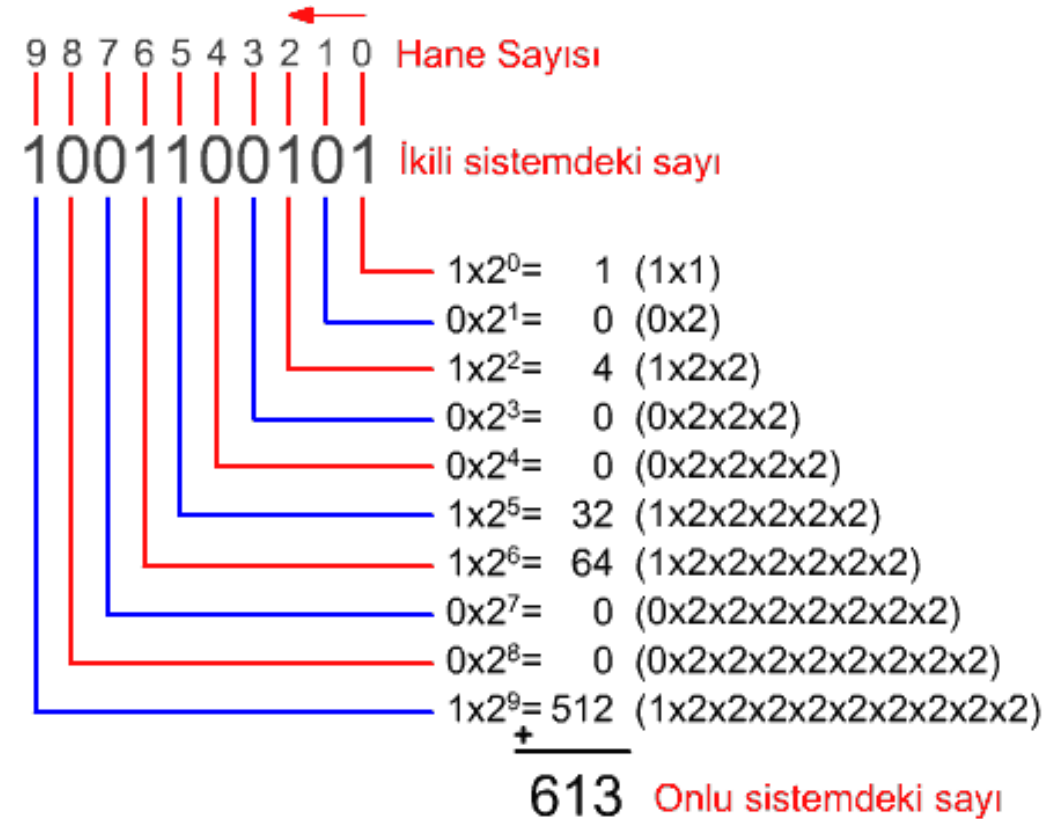
a. Base 10 system



b. Base two system



- İkilik sayıların 2 tabanında yazılmasıyla elde edilir. Dolayısıyla tüm sayılar 0 ve 1 kullanılarak ifade edilirler. Elektronik devrelerindeki kolay uygulanabilmeleri nedeniyle günümüz bilgisayarlarının neredeyse tamamında kullanılırlar.
- Bu sistemde her basamağın değeri, **2'nin kuvvetleri** şeklinde artar (... , 2^2 , 2^1 , 2^0).



İkilik Toplama

- İkilik toplama, ikilik sayı sistemindeki (0 ve 1) sayıların, elde (taşıma) kurallarına göre toplanmasıdır. Onluk toplamadaki mantık aynıdır; fark tabanın 2 olmasıdır.

İkilik toplama kuralları:

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 10 \rightarrow (0 \text{ yazılır, } 1 \text{ elde})$

	$\begin{array}{r} 11 \\ 1001101 \\ + 0010010 \\ \hline 1011111 \end{array}$	$\begin{array}{r} 11 \quad 1 \leftarrow \text{Carry bits} \rightarrow 11 \\ 1001001 \\ + 0011001 \\ \hline 1100010 \end{array}$	$\begin{array}{r} 11 \\ 1000111 \\ + 0010110 \\ \hline 1011101 \end{array}$
--	---	---	---

B I N A R Y A D D I T I O N

$$\begin{array}{r} + \quad 110 \\ \quad 111 \\ \hline 1101 \end{array}$$

$$\begin{array}{r} + \quad 10110 \\ \quad 10111 \\ \hline 101101 \end{array}$$

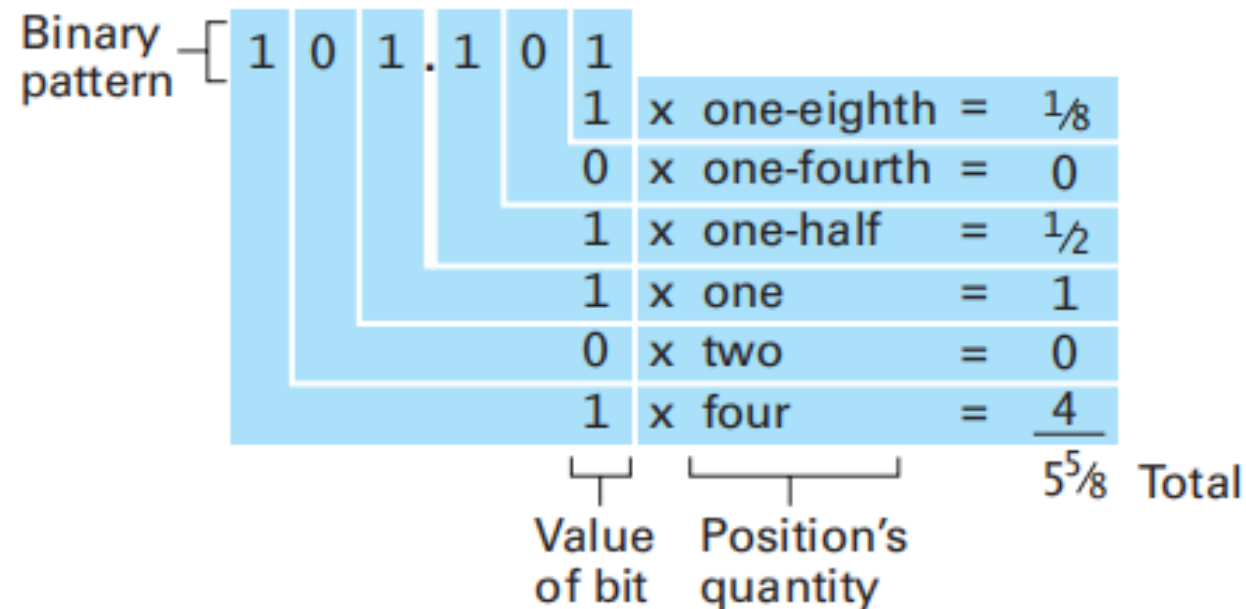
$$\begin{array}{r} \quad \quad 110101 \\ + \quad 100110 \\ \quad 100111 \\ \hline 10000010 \end{array}$$

İkilik Kesir

- Kesir değerlerini yerleştirmek amacıyla ikilik gösterimi genişletmek için, onluk gösterimde onluk noktası ile aynı görevde olan bir **taban noktası (radix point)** kullanırız. Bu noktanın solunda kalan rakamları değerın tamsayı kısmını (bütün bölümü) daha önceki sayfalarda anlattığım gibi ikilik sistemde olduğu gibi yorumlandığı anlamına gelir.

- Sağındaki basamaklar değerin kesirli kısmını ifade eder ve diğer bitlere benzer şekilde fakat pozisyonları kesirli büyüklüklere atanarak yorumlanmaktadır.

Noktanın sağındaki ilk pozisyon $\frac{1}{2}$ (2^{-1}), sonraki pozisyon $\frac{1}{4}$ (2^{-2}), sonraki $\frac{1}{8}$ (2^{-3}) olacak şekilde devam eden büyüklüklere atanmaktadır. Bit pozisyonlarına atanan bu değerler ile bir taban noktasına sahip bir ikilik gösterimin çözülmesi taban noktası olmayanda kullanılan aynı yöntemi gerektirmektedir. Daha doğrusu, gösterimde bitin pozisyonuna atanan büyüklük ile her bit değerini çarparız.

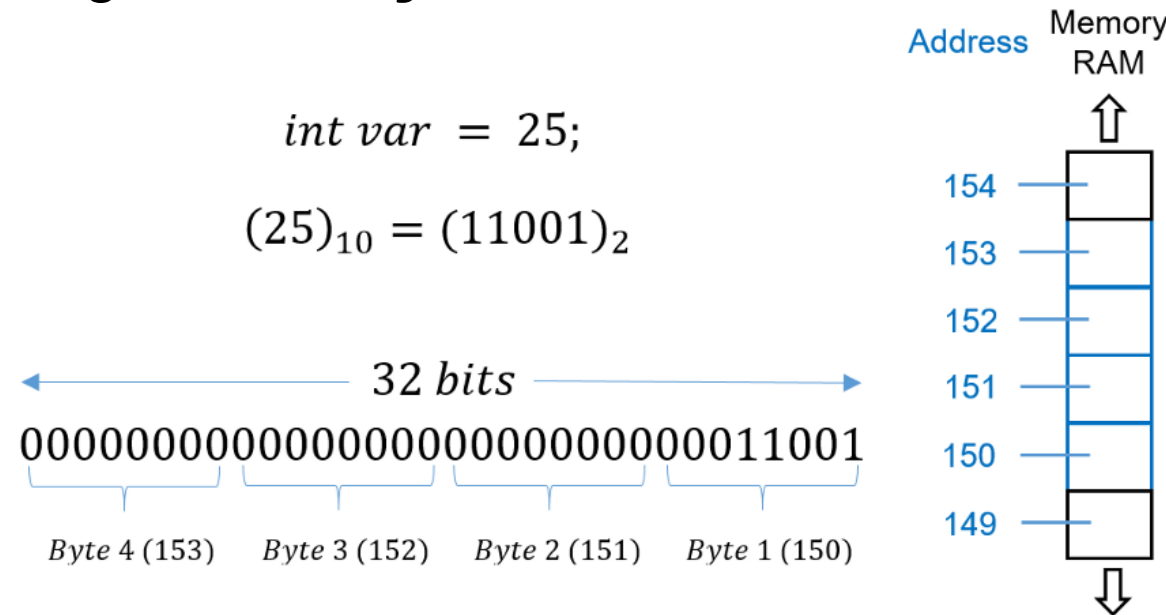


Tam Sayıları Depolama

- Tam sayıları depolama, bilgisayarda pozitif, negatif ve sıfır değerlerinin ikilik (binary) biçimde bellekte temsil edilmesi yöntemini ifade eder.

Temel noktaları:

- Bilgisayarlar yalnızca **0 ve 1** saklar.
- Tam sayılar **sabit sayıda bit** kullanılarak depolanır (ör. 8 bit, 16 bit, 32 bit).
- **En soldaki bit** genellikle **işaret bitidir**
- 0 → pozitif
- 1 → negatif



İkinin Tümüleyeni Gösterimi

Two's Complement

+

0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1

—

1	1	1	1	- 1
1	1	1	0	- 2
1	1	0	1	- 3
1	1	0	0	- 4
1	0	1	1	- 5
1	0	1	0	- 6
1	0	0	1	- 7
1	0	0	0	- 8

Sign-Bit

- Günümüzdeki donanımlarda, her değerin 32 bitlik bir desen ile gösterildiği bir ikinin tümleyeni sistemi yaygın olarak kullanılmaktadır. Negatif sayıları göstermek, toplama-çıkarma işlemlerini aynı donanımla yapabilmek, bilgisayarlarda aritmetiği basitleştirmek için kullanılır. Böyle büyük bir sistem geniş yelpazede sayıları göstermeyi sağlamaktadır, fakat gösterim şekli hantaldır. Bu nedenle, ikinin tümleyen sistemi özelliklerini çalışmak için, daha küçük sistemler üzerinde yoğunlaşacağız.

- İkinin tümleyeni gösteriminde, bir bit deseninin en soldaki biti gösterilen değerin işaretini belirttiğini hatırlayalım. Bu nedenle, en soldaki bit genellikle **işaret biti** olarak adlandırılmaktadır. Bir ikin tümleyen sisteminde, negatif bitler işaret bitleri 1 olan desenler tarafında gösterilmektedir; negatif olmayan değerler ise işaret biti 0 olan desenler tarafından gösterilmektedir.

$$\begin{array}{rcl}
 +34 & = & 0\ 0\ 1\ 0\ 0\ 0\ 1\ 0 \\
 -34 & = & 1\ 0\ 1\ 0\ 0\ 0\ 1\ 0
 \end{array}
 \left|
 \begin{array}{l}
 1111\ (-1) \\
 +0001\ (+1) \\
 \hline
 \text{-----}\ (0)
 \end{array}
 \right.$$

- Bu temsil yöntemi, sıfırın tek bir gösterime sahip olmasını sağlar ve taşma (overflow) durumlarının tamamını netleştirir. Bu özellikler nedeniyle ikinin tümleyeni sistemi, hem donanım hem de yazılım seviyesinde güvenilir ve yaygın olarak tercih edilir.

Taşma Problemi (Overflow)

$$\begin{array}{r} 01111000 \\ + 00111110 \\ \hline 10110110 \end{array}$$

Incorrect Sign

Incorrect Magnitude

$$\begin{array}{r} 120 \\ + 62 \\ \hline 182 \end{array}$$

- Taşma, bir hesaplamada gösterilebilecek değerler aralığının dışına düşen bir değer ürettiğinde ortaya çıkan bir problemdir. İkinin tümleyeni gösterimi kullanıldığında, bu iki pozitif değer toplandığı zaman ya da iki negatif değer toplandığı zaman ortaya çıkabilmektedir. Her iki durumda da, bu durum cevabın işaret biti kontrol edilerek tespit edilebilmektedir. Taşma, iki pozitif değer toplamalarının negatif bir değer deseni göstermesi ile sonuçlanıyorsa ya da iki negatif değer toplamı pozitif görünüyorsa, ortaya çıkmaktadır.

Bit pattern	Value represented
1111	7
1110	6
1101	5
1100	4
1011	3
1010	2
1001	1
1000	0
0111	-1
0110	-2
0101	-3
0100	-4
0011	-5
0010	-6
0001	-7
0000	-8

Fazlalık (Excess/Biased) Gösterimi

- Fazlalık (excess) gösterimi, işaretli sayıları temsil etmek için kullanılan bir sayı gösterim sistemidir. Bu yöntemde gerçek sayı, önceden belirlenmiş bir sabit değer (bias/fazlalık) eklenerek saklanır. Yani bellekte tutulan değer, gerçek sayı değil; gerçek sayının kaydırılmış (offset'li) hâlidir.

- Temel fikir şudur;
 - ✓ Bir sayı depolanmadan önce:

$$\text{Saklanan Değer} = \text{Gerçek Değer} + \text{Bias}$$

- ✓ Okunurken ise:

$$\text{Gerçek Değer} = \text{Saklanan Değer} - \text{Bias}$$

Bu nedenle tüm sayılar işaretli (signed) gibi saklanır.

Bias (Fazlalık) Nedir?

- Bias, genellikle bit sayısına bağlı olarak seçilir.

n bit için en yaygın bias:

$$\text{Bias} = 2^{(n-1)}$$

Örnek (4 bit, Excess-8)

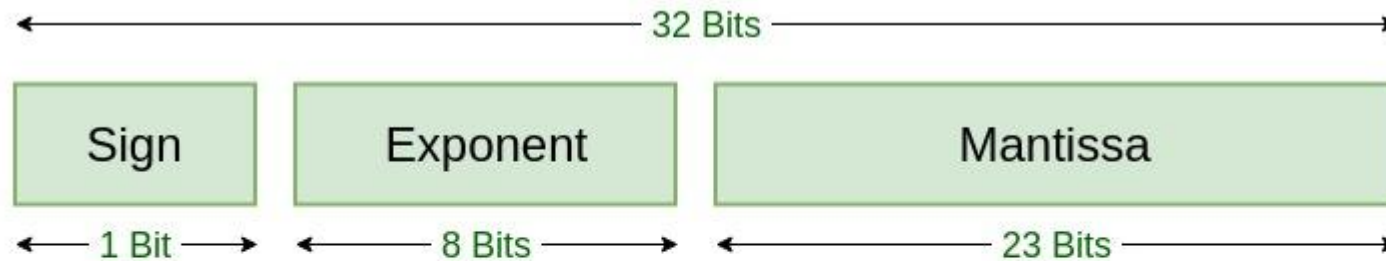
4 bitlik bir excess gösteriminde:

$$\text{Bias} = 8$$

GERÇEK DEĞER	SAKLANAN DEĞER	İKİLİK GÖSTERİM
-8	0	0000
-1	7	0111
0	8	1000
+3	11	1011
+7	15	1111

Nerede Kullanılır?

- Karşılaştırma işlemleri çok kolaydır. (Unsigned karşılaştırma ile doğru sonuç verir.)
- Donanımda işaret biti gerekmez.
- Özellikle kayan noktalı sayı (floating-point) sistemlerinde tercih edilir.



Single Precision
IEEE 754 Floating-Point Standard

KAYNAKÇA

<https://chatgpt.com>

<https://medium.com>

<https://tr.wikipedia.org>

<https://www.tech-worm.com>

<https://evrimagaci.org>

Computer Science- An Overview (12th Global Edition)

<https://uzman-bilgisayar.com>

<https://www.cagataycebi.com>

<https://mrdorancomputing.com>

<https://diyot.net.com>

<https://www.allaboutcircuits.com>

<https://www.allaboutelectronics.org>

<https://sumitanantwar.com>